Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Boot Loader Application

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date	
Prepared :	James Kim Software Designer	James Kim	2011-09-07
Verified :	Eric Malo Eng. System Engineer		2011-09-12
Approved :	Martin Ouellet Eng. Software Department Supervisor		2011-09-12

Document No.	**201111456**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-08-26**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-08-26	James Kim	Official release
0.1	---	2011-08-25	James Kim	Implementing inspection comments from the inspection form I-1743.
0	---	2011-08-09	James Kim	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and 1558-2004.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 6

1 INTRODUCTION 7

1.1 Purpose 7
1.2 Scope 7
1.3 Definitions and Acronyms 7
1.3.1 Definitions 7
1.3.2 Acronyms 7
1.4 Document Organization 8

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition Description 10
3.1.1 Entities Decomposition 10
3.1.2 Data Decomposition 11
3.2 Dependency Description 13
3.2.1 Inter-Entity Dependencies 13
3.2.2 Inter-Process Dependency 13
3.2.3 Data Dependencies 14
3.3 Interface Description 15
3.3.1 Module Interfaces 15
3.4 Detailed Design 19
3.4.1 Entities Detailed Design 19
3.4.2 Data Entities Detailed Design 41
3.5 Operating System Services 41
3.6 Memory Layout 41

APPENDIX A: FAULT LED BLINKING PATTERNS 42

APPENDIX B: SPECIFIC PROJECT DESIGN ELEMENT 43

List of Figures

Figure 3-1: Inter-Entity Dependencies Diagram 13
Figure 3-2: Inter-Process Dependency Diagram 13
Figure 3-3: Boot_Loader Module Flow Chart 20
Figure 3-4: HW_COM Port Block Diagram 21
Figure 3-5: SERIAL_EncodePacket Flowchart 22
Figure 3-6: SERIAL_TaskPacketDecode Flow Chart 24
Figure 3-7: PTA_Task flow chart 25
Figure 3-8: PTA Login Request 28
Figure 3-9: WDT_Refresh flow chart 30
Figure 3-10: FaultLEDutility Flow Chart 31
Figure 3-11: IRAM InternalRAMCheck flow chart 33
Figure 3-12: RAM_RamCheck flow chart 36
Figure 3-13: CRC_CodeIntegrity flow chart 39
Figure 3-14: FLASH_EraseAppCode Flow Chart 40
Figure 3-15: FLASH_WritePacket Flow Chart 40

List of Tables

Table 3-1: Structure of MemBackupZone for pattern and address test verification 11
Table 3-2: Structure Code Info 11
Table 3-3: Packet Header Type 23
Table 3-4: Command Implemented in Boot Loader 26
Table 3-5: File Index Number and File Type 26
Table 3-6: Index of the PTA_SciVersion 27
Table 3-7: Codes of the Login and Logout 28
Table 3-8: Index of the CodeStatusRequest 29
Table 3-9: Command codes sent to Fault LED Utilities 30
Table 3-10: Memory Test Structure 34
Table 3-11: Function to calculate the CRC16 37
Table 3-12: CRC Look up Table for CRC16 calculation 37
Table 3-13: Status Codes returned for CRC Validation Module 38

1 INTRODUCTION

This document is the Software Design Description of the SD1 Boot Loader Application developed by Vapor Stone Rail Systems.

The Boot Loader Application is part of the Microprocessor Based System (MBS) responsible for performing basic hardware verification and allowing application upgrades.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the generic software components and required implementation details of the Boot Loader Application.

The intended audience is the Vapor Stone Rail Systems software development team, maintenance team members of the transit authority and the car builder.

1.2 Scope

The Boot Loader is a software that resides inside any module of the Door Module System (DMS). It is the startup software of the Door Module System controller that verifies the integrity and initializes the processor board. The Boot Loader enables no door operation.

This document describes the generic design of the Boot Loader. The complete design of the Boot Loader is defined in two documents: this generic document and the Software Applications Configuration Specifications (SACS) document.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Module System:	A processor based electronic box running software that controls the door, provides monitoring functions, and interacts with the user via a PTA and a network.
Boot Loader:	Software that verifies the CPU and memory, activates the Warning and Fault Light Emitting Diodes (LED), communicates with the PTA and starts the main application.
Portable Test Application (PTA)	A software tool used by maintenance and engineering personnel to retrieve logs, do real time monitoring, change door parameters, and upgrade to new application. It runs on Portable Test computer and connects to DMS via Hardware Communication Interface (HW_COM).

1.3.2 Acronyms

ASCII	American Standard Code for Information Interchange
CCITT	Consultative Committee for International Telephone / Telegraph
CDRL	Contract Data Requirements List
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CRC16	16-bit Cyclic Redundancy Check
DMS	Door Module System
ECD	Engineering Change Document
EOM	End Of Message
HW_COM	Hardware Communication Interface
ICD	Interface Control Document
IEEE	Institute of Electrical and Electronics Engineers
IRAM	Internal Random Access Memory
KB	Kilobytes
LED	Light Emitting Diode
MHz	MegaHertz

MBS	Microprocessor Based System
ms	millisecond
N/A	Not available
NVRAM	Non-Volatile Random Access Memory
OS	Operating System
PEC	Peripheral Event Controller
PTA	Portable Test Application
RAM	Random Access Memory
RS232	Recommended Standard 232 (Computer Serial Interface)
RTC	Real-Time Clock
RTS	Request To Send
SACS	Software Applications Configuration Specifications
SCI	Software Configuration Items
SDD	Software Design Description
SFR	Special Function Register
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TS	Technical Specification
Tx	Transmission
VSRS	Vapor Stone Rail Systems
XRAM	On-Chip Extension Random Access Memory

1.4 Document Organization

This SDD is divided into eight sections and an appendix with the following specific information:

- Section 1, Introduction states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interaction between entities.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.
    - Operating System Services: mentions the services used in the Real-Time Operating System.
    - Memory Layout: shows Flash and RAM memory mapping.
- Appendix A, Fault LED Blinking Pattern, enumerates Fault LED patterns.

Within the document, diagrams, flowcharts and tables are used to present the Boot Loader Application design:

- Figure 3-4 details HW_COM port function parts.
- Decision flowcharts give a graphical view of how entities and functions process work.
- Data flow diagrams Figure 3-1 describes the dependency between entities.
- Tables contain data structures used by the Boot Loader. Examples are test RAM and SCI information structures. They also present C code of short functions and CRC calculation look-up tables.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

201111127	Software Requirement Specification, – Boot Loader Application, Rev. 1
201111489	Interface Control Document - Portable Test Application, Rev. 1
S17-07	Software Design Description Template, Rev. 0
N/A	Software Applications Configuration Specifications, Rev. N/A

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Entities Decomposition

3.1.1.1 Boot Loader Module

Boot Loader Module initializes the CPU and its peripherals (User stack, Watchdog Timer, etc.).
The Boot Loader starts the Volatile and Non-Volatile Integrity Check. Then it starts the CRC verification on the Boot Loader code.
The Boot Loader Module starts the CRC verification of the application code. If the PTA is connected to the DMS or the CRC on the Application code doesn't match, it starts the PTA Command Interpreter. Otherwise the Boot Loader Module terminates the tasks and starts the Application.

3.1.1.2 Serial Module

The Serial Module takes in charge to decode and parse PTA packets received on the serial port.

3.1.1.3 PTA Boot Module

The PTA Boot Module is responsible to communicate with the PTA in the application layer.

3.1.1.4 Watchdog Module

The Watchdog Module takes care of the Watchdog peripheral. It periodically updates the timer in order to avoid reset of the Door Module System. This Module also flashes the Warning LED.

3.1.1.5 LED Utility Module

This LED Utility Module is responsible for the flashing rate of the Fault LED depending on the error code.

3.1.1.6 Memory Integrity Utility Module

The Memory Integrity Utility Module verifies the integrity of the Volatile and Non-Volatile memories before the program can access it.

3.1.1.7 CRC Utility Module

The CRC Utility Module is responsible for verifying the integrity of the Boot Loader and the application. It also provides the CRC calculation utility.

3.1.1.8 Flash Utility Module

This Module contains all services needed to identify the type and size of flash memory installed, to write, read, erase sectors, and compare written data.

3.1.2 Data Decomposition

3.1.2.1 Memory Backup Zone

This section shows the structure used by Memory Integrity Utility Module to backup the memory location when performing test in Volatile and Non-Volatile Memories. There is a 2 parts union. The first part is used when the address lines test is in progress, to backup data on memory cells to be tested. The second part of the union is used when the pattern test is in progress.

Table 3-1: Structure of MemBackupZone for pattern and address test verification

Identification	MemBackupZone
Entity Type	struct RamTest

Prototype	`typedef union`
	`{`
	`  struct`
	`  {`
	`  unsigned char ucNotUsed1;`
	`  unsigned char ucCommand;`
	`  unsigned char ucComplCommand;`
	`  unsigned char ucNotUsed2;`
	`  unsigned int  uiCrc;`
	`  unsigned char ucNbData;`
	`  unsigned char ucData[22];`
	
	`  }AddressLine;`
	
	`  struct`
	`  {`
	`  unsigned char ucNotUsed1;`
	`  unsigned char ucCommand;`
	`  unsigned char ucComplCommand;`
	`  unsigned char ucNotUsed2;`
	`  unsigned int  uiCrc;`
	`  unsigned long ulAddress;`
	`  unsigned int  uiData;`
	
	`  }Pattern;`
	
	`}RamTest;`
	
	`RamTest far MemBackupZone;`

3.1.2.2 Code Info Structure

Table 3-2 shows the structure used to get the code information stored in Flash for both SCIs: Application and Boot Loader.

Table 3-2: Structure Code Info

Identification	INFO_BOOTLOADER and INFO_APPLICATION
Entity Type	struct INFO_SCI

Prototype	`#define SCI_NAME_SIZE  8  // Size of SCI name in the ucAcronym variable.`
	`#define SCI_REV_SIZE  6  // Size of the revision string.`
	`#define SCI_APPLICATION  1`
	`#define SCI_BOOT_LOADER  4`
	`#define SCI_TYPE_SOFTWARE_VERSION 1  // #1 => SCI software can be updated`
	`  // directely on equipement.`
	
	`typedef struct _INFO_SCI`
	`{`
	`  unsigned long ulCodeSize;  // CODESIZE.`

```
    unsigned int  uiCodeCrc32;
    unsigned char ucSciIndex;
    unsigned char ucAcronym[SCI_NAME_SIZE];
    unsigned char ucRevision[SCI_REV_SIZE];

    union _CompilationDate
    {
        unsigned char ucCompilDate[11]; // Date of compilation of the SCI.
        INFO_Date Date;
    } CompilDate;

    unsigned char ucSpare;              // Spare byte
    union _CompilationTime
    {
        unsigned char ucCompilTime[8];  // Time of compilation of the SCI.
        INFO_Time Time;
    }CompilTime;

    unsigned int  uiOSVersion;    // Version of the OS.
    unsigned char ucReserved[4];  // Field reserved fo futur use.

}INFO_SCI;

typedef struct INFO_Date
{
    unsigned char   ucMonth[3]; // Revision Month Mmm in ASCII, ex “Oct”
    unsigned char   ucSpace1;
    unsigned char   ucDay[2];   // Revision day DD in ASCII, ex: “30”
    unsigned char   ucSpace2;
    unsigned char   ucYear[4];  // Revision year YYYY in ASCII, ex: “2004”

}INFO_Date;


typedef struct INFO_Time
{
    unsigned char   ucHour[2];
    unsigned char   ucSpace1;
    unsigned char   ucMinute[2];
    unsigned char   ucSpace2;
    unsigned char   ucSeconds[2];

}INFO_Time;

INFO_SCI const near INFO_DCU_BOOTLOADER =  {0xFFFFFFFF, 0xFFFF,
SCI_BOOT_LOADER, SCI_TYPE_SOFTWARE_VERSION, "BOOT", "1.x", __DATE__,0,
_TIME_,OS_VERSION};

INFO_ SCI const far INFO_DCU_APPLICATION;
```

3.2 Dependency Description

3.2.1 Inter-Entity Dependencies

Figure 3-1 shows the Inter-Entity Dependencies Diagram of the Boot Loader.

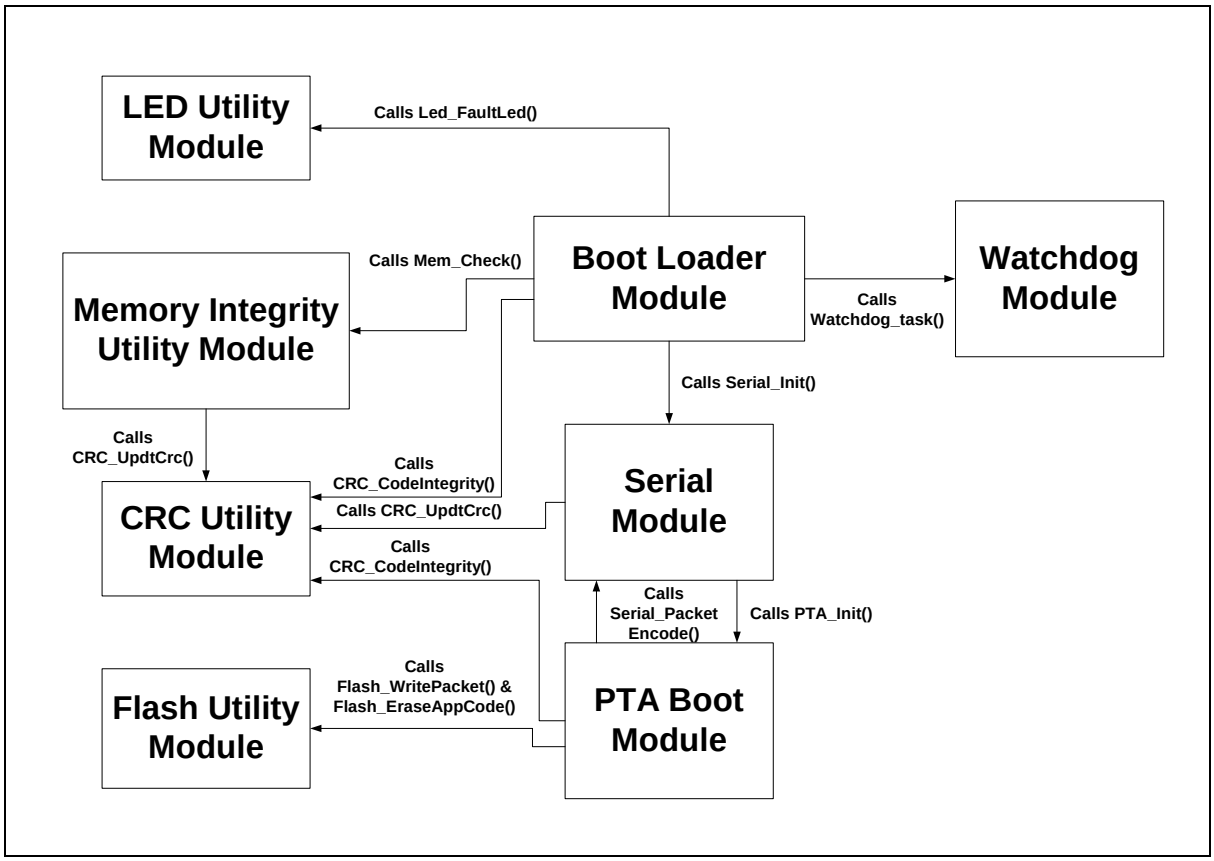

Figure 3-1: Inter-Entity Dependencies Diagram

3.2.2 Inter-Process Dependency

The Serial and PTA Boot modules communicate through the OS mail box. When the Serial Module receives the PTA packets from the PTA, it extracts the protocol data from the packets and sends the mail to PTA Boot Module. Once the PTA Boot Module receives the mail, it proceeds to prepare the response.

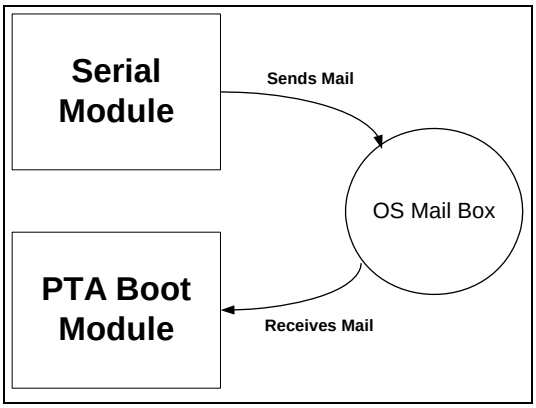

Figure 3-2: Inter-Process Dependency Diagram

3.2.3 Data Dependencies

The PTA Boot and the CRC Utility Modules retrieve the code information data of the Boot Loader and Application. The structure of these data is shown in the Table 3-2: Structure Code Info, and the address of the code information for the Boot Loader and Application are defined in the specific project's *Software Applications Configuration Specifications (SACS)*.

3.3 Interface Description

This section describes the functions of each module of the Boot Loader. The function name, a description of the function, input and output parameters required by the function, and the name of the module that called the function are provided for each function of each module.

3.3.1 Module Interfaces

3.3.1.1 Boot Loader Module Interface

Function Name	void BOOT_LoaderModule (void)
Description	The Boot Loader Module initializes the CPU and peripherals, call test modules and starts the Application if present and valid.
Input	None.
Output	None.
Return	None.

Function Name	void BOOT_Task (void)
Description	The Boot Loader task creates the Watchdog task and calls the Serial_Init. Then it destroys itself.
Input	None.
Output	None.
Return	None.

3.3.1.2 Serial Module Interfaces

Function Name	void SERIAL_Init (int iFcpu, unsigned long ulBaud, unsigned int uiSetting)
Description	This function initializes the Serial component. It effects all necessary configurations of the SFR, depending on the selected operating mode. The configuration determines whether the Serial interrupts are to be released, and the priority of the released interrupt.
Input	iFcpu: Frequency of the CPU board in MHz. ulBaud: Baud rate of the Serial Port. uiSetting: Port setting. For example 8-N-1 = DATA_BIT_8 | IGNORE_PARITY | STOP_BIT_1.
Output	None.
Return	None.

Function Name	void SERIAL_TaskPacketDecode (void)
Description	This task waits for the semaphore sent by SERIAL_TaskStringRx. It validates the content of the received message and sends it to PTA_Command Interpreter. In case the received message contains errors, a message is sent to DmsToPtaError to notify the PTA that the received message contains errors.
Input	None
Output	None.
Return	None.

Function Name	void SERIAL_TaskStringRx (void)
Description	This task takes the character present in the reception buffer and copy them into the working buffer. Pointers of the PEC Channel 0 are then reset. This task runs every 5 ms.
Input	None.
Output	None.
Return	None.

Function Name	bit SERIAL_EncodePacket (unsigned char ucLength, unsigned char xhuge *pucBuf)
Description	This function takes the data to be transmitted and encodes it. This function is the layer 2 and 3 of the transmission process. The final packet is transferred to the transmit buffer for sending.
Input	ucLength: Data length. pucBuf: Pointer to the beginning of the message to encode.
Output	None.
Return	This function returns TRUE if the string to send fits into the TX buffer. It returns FALSE if the string to send is too large for the TX buffer.

3.3.1.3 PTA Boot Module Interface

Function Name	void PTA_TaskCommandInterpreter (void)
Description	This task waits for the mail from the SERIAL_TaskPacketDecode. Once the mail is received, it manages the PTA communication.
Input	None.
Output	None.
Return	None.

Function Name	bit PTA_Init (void)
Description	This function initiates the OS mail box to be used and creates the task for the PTA Boot Module.
Input	None.
Output	None.
Return	This function returns TRUE if the task is created successfully. Otherwise it returns FALSE.

3.3.1.4 Watchdog Module Interface

Function Name	void Watchdog_Task (void)
Description	This function refreshes the Watchdog Timer by triggering the internal hardware watchdog of the CPU and toggles the Warning LED state every 250 ms. The period should be within the reset period of the Reset & Watchdog Timer device (~350 ms at 24 MHz).
Input	None.
Output	None.
Return	None.

3.3.1.5 LED Utility Module Interface

Function Name	void Led_FaultLed (char cStatusCode)
Description	This function displays the fault status. In case the module is called to display the status, the module turns the LED “on” or “off”. In case the module is called to display an error code with the LED, the module never returns to the function caller and runs indefinitely to flash the LED with the appropriate flashing pattern. In that case, the HW_COM port and interruptions are deactivated.
Input	cStatusCode: Fault status to display on LED.
Output	None.
Return	None.

3.3.1.6 Memory Integrity Utility Module Interfaces

Function Name	char Mem_Check (void)
Description	Before initiating the memory tests, this function verifies if a previous test was in progress and replaces the values. The new data are then backed up before test starts. The memory is tested with 2 tests; the first test is to verify that all address lines to the memory are working properly, and the second test is to verify that all bits of the memory are working by writing a pattern data AA55h and 55AAh. The return value is a Boolean that indicates the status of the operation.
Input	None.
Output	None.
Return	This function returns Success (1) or Fail (0).

Function Name	void IRAM_InternalRamCheck (void)
Description	This module is written in assembly language because it should be executed before any initialization of CPU, interfaces, and before the C startup code is executed. Each section is tested with 2 tests; the first test is to verify that all address lines to the memory are distinct and working properly, and the second test is to verify that all bits of the memory are working by writing a pattern data AA55h and 55AAh. The fault led will blink once at each second if the test failed. At the end of this module, a jump to the INIT chip module is performed if there is no error.
Input	None.
Output	None.
Return	None.

3.3.1.7 CRC Utility Module Interfaces

Function Name	unsigned char CRC_CodeIntegrity (unsigned char xhuge *pucStartingAddress)
Description	This function computes the CRC on data in a location of the flash memory. The return value is the status of the comparison of the calculated CRC with the one stored in Flash memory.
Input	*pucStartingAddress: Contains the address of the section of the flash memory to calculate the CRC.
Output	None.
Return	This function returns the result of the comparison (NO_ERROR, CODE_NOT_PRESENT, CRC_NO_MATCH, OTHER_ERROR).

Function Name	unsigned int CRC_UpdtCrc (unsigned char ucNewData, unsigned int uiOldCrc)
Description	This function takes the input data value and calculates its CRC16 value.
Input	ucNewData: The new data to add on the old CRC16. uiOldCrc: The old CRC16 value to add.
Output	None.
Return	This function returns the calculated CRC16 value in unsigned integer.

3.3.1.8 Flash Utility Module Interfaces

Function Name	bit Flash_EraseAppCode (unsigned long ulSectorBase)
Description	This module is used when a new Application code is programmed into the Flash Memory section reserved for the Application. This module should be located and executed from the internal RAM or the static RAM. This module erases the sections related to the Application code.
Input	ulSectorBase: Base of the sector to erase in the Flash.
Output	None.
Return	This function returns the bit value bStatus; it returns True for success, and False for fail.

Function Name	bit Flash_WritePacket (volatile unsigned int xhuge *puiDestAddr, volatile unsigned int xhuge *puiSourceAddr, unsigned long ulNumberOfBytes)
Description	This module is used when a new Application code is programmed into the Flash Memory section reserved for the Application. This module should be located and executed from the internal RAM or the static RAM. This module receives from the PTA Command Interpreter the data and programs it into the Flash memory.
Input	*puiDestAddr The pointer to the destination address on the Flash. *puiSourceAddr The pointer to the source data. ulNumberOfBytes The number of bytes of the data.
Output	*puiDestAddr The destination on the Flash where the next packet will be written.
Return	This function returns the bit value bStatus; it returns True for success, and False for fail.

3.4 Detailed Design

This section describes how the Boot Loader works and what operation it performs. Each task executed by the Boot Loader is a sequential process. The PTA Command Interpreter is described in section 3.4.1.3.

3.4.1 Entities Detailed Design

3.4.1.1 SD1.1 Boot Loader Module

During start-up, the Boot Loader:

- Initializes the CPU,
- Loads configuration,
- Configures serial port,
- Verifies integrity of CPU Internal RAM (performs pattern test),
- Verifies integrity of RAM (performs pattern test and address test),
- Verifies integrity of the Flash memory for the Boot Loader section (performs CRC),
- Verifies presence and integrity of Application (performs CRC),
- Calls the SERIAL_Init to create the Serial task.

Figure 3-3 shows the flowchart of the Boot Loader. At the start-up, the first operation is the system initialization of the basic functions on the CPU that the Boot Loader uses. The Warning LED is turned ON to display status of the DMS. See Appendix A for explanation of meaning of Warning and Fault LED codes.

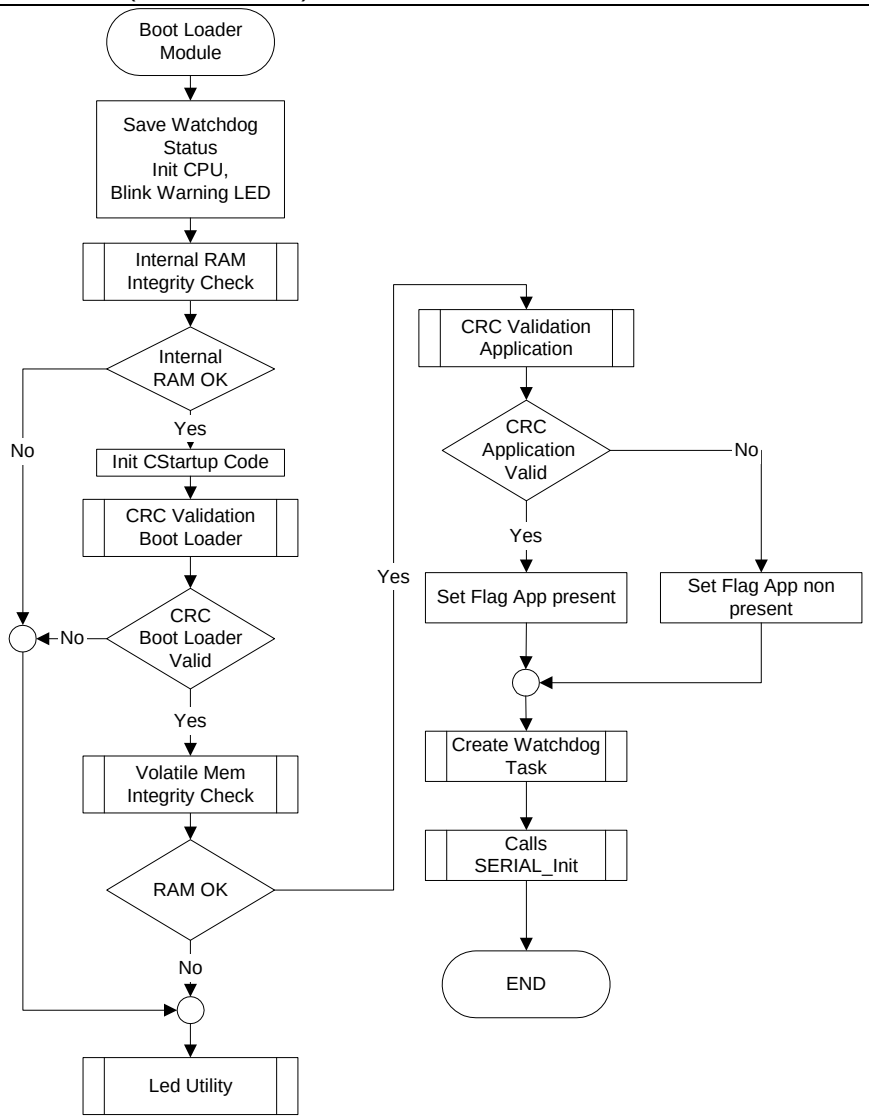

Figure 3-3: Boot_Loader Module Flow Chart

When the CPU used is ST10F276, the system verifies the integrity of the Internal RAM before the Boot Loader can use it. If the test fails, the system starts to blink the Fault LED with the associated fault code until a hardware reset on the CPU is performed. Otherwise, it skips the test.

The next step is the initialization of the other peripherals; the registers installed in the Internal RAM and the start-up code for the C interpreter.

Then the system calculates the CRC on the Boot Loader to verify the integrity of the code written in Flash Memory. In the case that the calculated CRC doesn't match with the one stored in Flash Memory, the system start to blink the Fault LED with the associated fault code until a hardware reset on the CPU is performed.

If the CRC matches, the system performs a verification of the Volatile memory. If the test fails, the system starts to blink the Fault LED with the associated fault code until a hardware reset on the CPU is performed.

The Boot Loader verifies the presence and the validity of the Application code in Flash Memory. If the Application is not present, the Boot Loader sets the flag "Application Non-Present".

If the Application is present in Flash Memory, the CRC of the Application is calculated and compared to verify the integrity of the Application code written in Flash Memory. In the case that the CRC verification fails, the Boot Loader sets the flag “Application Non-Present” as well.

After the verification, the Boot Loader calls the SERIAL_Init to create the Serial tasks.

3.4.1.2 SD1.2 Serial Module

There are two main functions in the Serial Module: Packet Encoder (SERIAL_EncodePacket) and Packet Decoder (SERIAL_TaskPacketDecode).
The Packet Encoder is the function that prepares the message to be transmitted. It calculates the CRC, adds the header and adjusts the message length. It copies the message to be transmitted into the TX_BUF and adjusts the pointers. If a transmission was not in progress when the message is copied, the transmission is then started.
The Packet Decoder verifies if a message is received and complete into the RX_BUF.
The Packet Encoder and Decoder reads the CPU input pin to determine if the controller is the left or right hand side. By default, the value of the Controller ID is zero. The SERIAL_EncodePacket function inserts the Controller ID value to the outgoing message of the PTA communication.

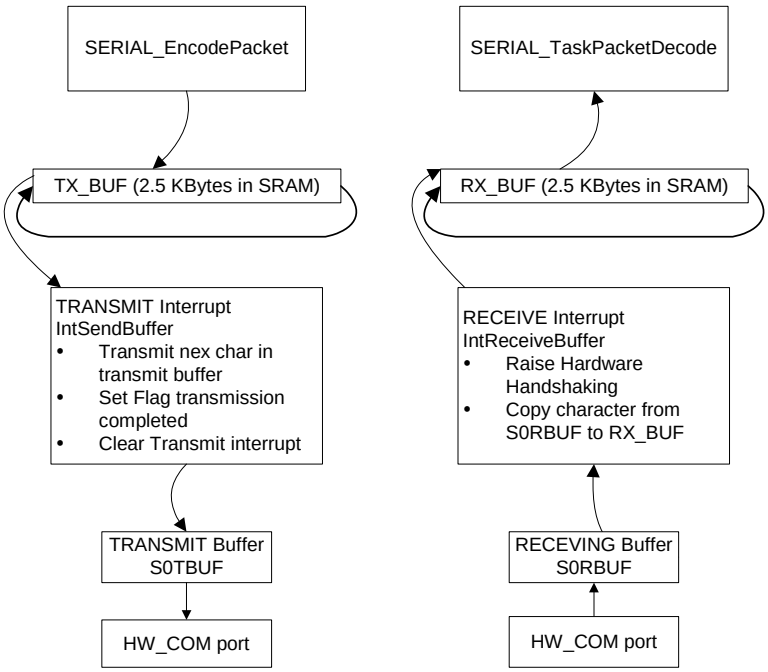

Figure 3-4: HW_COM Port Block Diagram

3.4.1.2.1 SD1.2.1 SERIAL_EncodePacket Function

This function is called from the PTA Command Interpreter with a command and its associated data to be transmitted. The command can be a re-sent of the last packet command or a new packet. The message to be transmitted contains the header and the data. If there is a data in the message, the function adds the EOM, calculates the CRC and adjusts the message length. The pointer in TX_BUF is adjusted. If there was no transmission in progress, it starts the transmission of the message.

In the case where the command is to be resent, the pointer in TX_BUF is readjusted. If there was no transmission in progress, it starts the transmission of the message. Figure 3-5 shows the flowchart of the SERIAL_EncodePacket function.

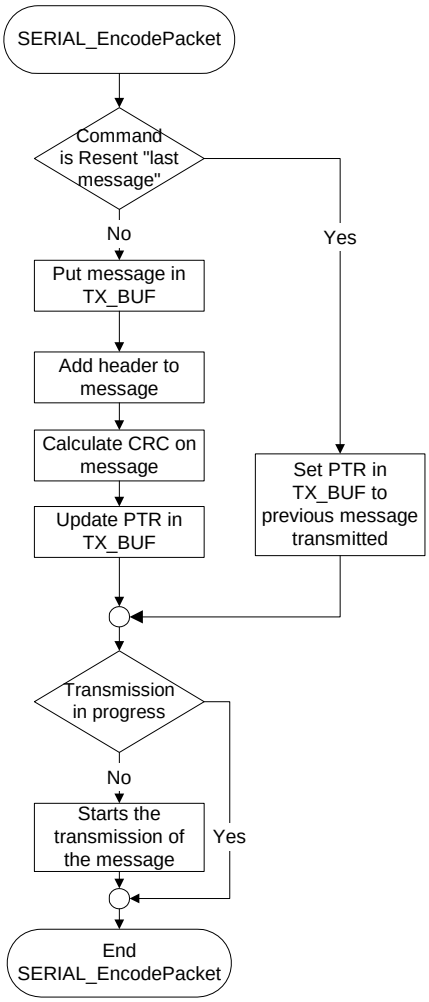

Figure 3-5: SERIAL_EncodePacket Flowchart

3.4.1.2.2 SD1.2.2 SERIAL_TaskPacketDecode Function

This function is a task that runs in a loop. It verifies if characters are present in the RX_BUF. It also verifies if a packet is completed. It verifies if the first character received is a recognized packet header. See Table 3-3 for the type of framing character recognized by the decoder. If the first byte is an OEM command, the decoder verifies if the next byte is the EOM character. If the command received is a “PASS TOKEN COMMAND”, the function returns to the PTA_CommandInterpreter and indicates to the Boot Loader that it can send messages or pass back the token to the PTA.

If the first byte is CMD_XFER, the next byte in the buffer is fetched and memorized to know the length of the message. If the complete message is not received within the timeout delay of 100ms, the received bytes are then discarded and the pointers are reset to allow reception of new messages.

When the message is received within the timeout delay, a validation of the CRC is performed. If the CRC doesn’t match the one calculated, the message is then discarded and the pointers are reset to allow reception of new messages. A flag is set to indicate that the message received has a bad CRC and the PTA_CommandInterpreter is responsible to sends the PTA_DMStoPTError message.

In case the calculated CRC match with the one received, a flag is set to indicate that the message received is valid, a flag is set to let the PTA_CommandInterpreter take care of the message.

Table 3-3: Packet Header Type

Code	Name
0x01	CMD_XFER
0x00	EOM

Figure 3-6 shows the flowchart of the SERIAL_TaskPacketDecode function.

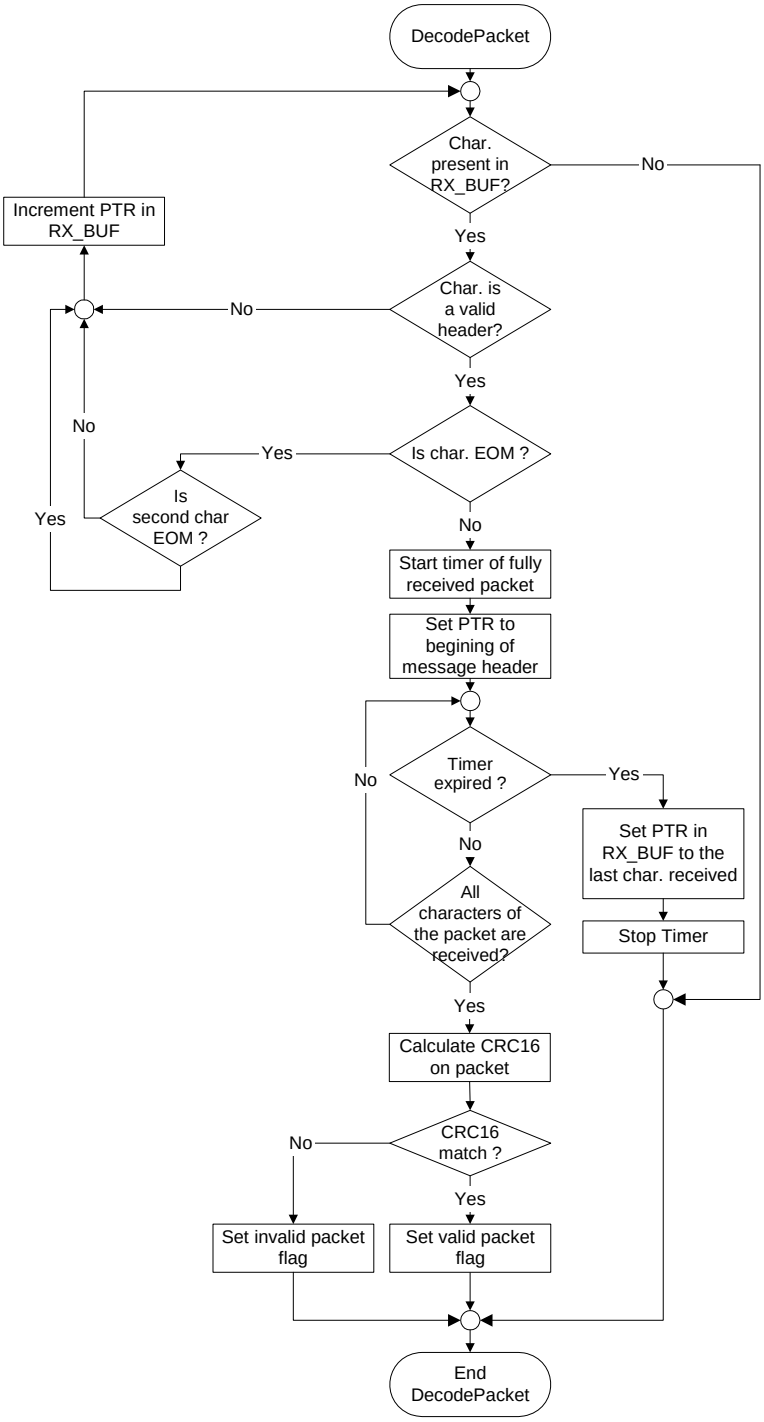

Figure 3-6: SERIAL_TaskPacketDecode Flow Chart

3.4.1.3 SD1.3 PTA Boot Module

3.4.1.3.1 SD1.3.1 PTA Command Interpreter

The Boot Loader PTA Command Interpreter is a task that handles the communication with the PTA.
The PTA Command interpreter task waits for the command described in the PTA Interface Control Description (ICD) document #201111489. There is only a sub set of command that the PTA Command Interpreter of the Boot Loader can execute.

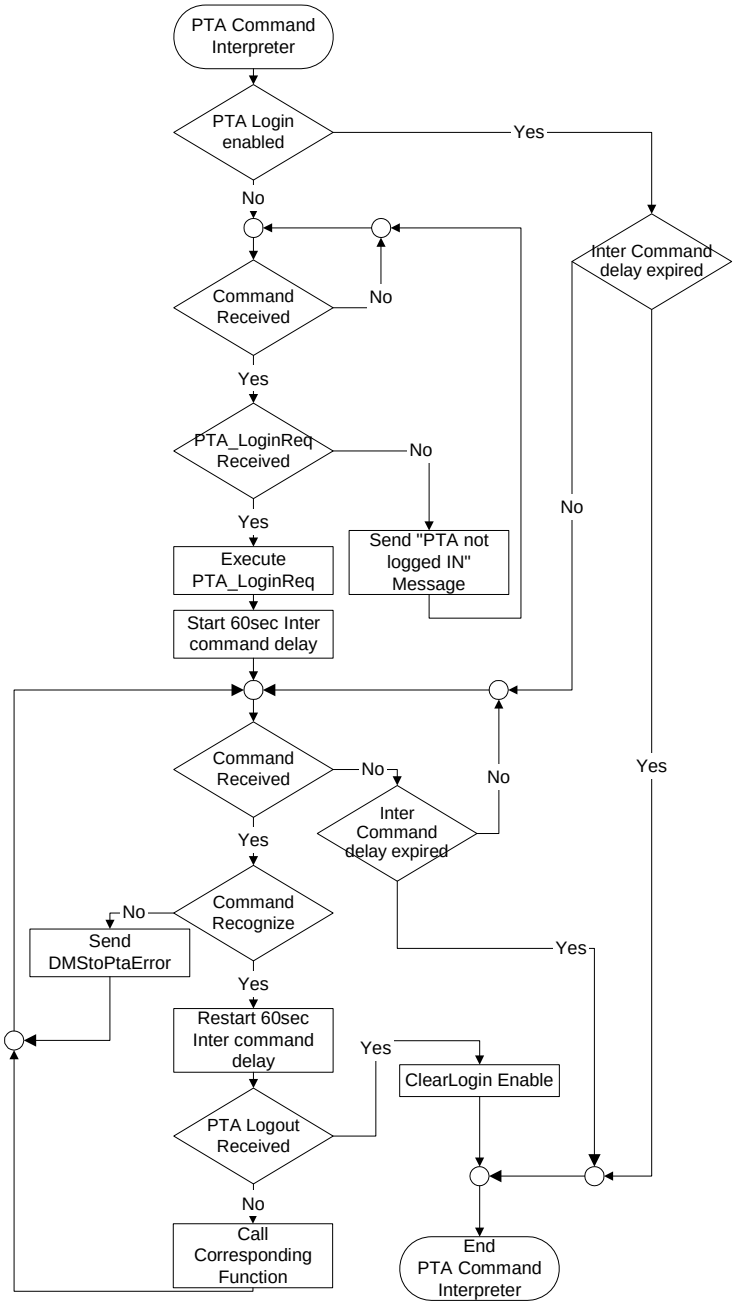

Figure 3-7: PTA_Task flow chart

Table 3-4: Command Implemented in Boot Loader

HW_COM Communication	HW_COM Elements	Portocol Code	Function Name
Login	LoginRequest	70	PTA_LoginReq
	LoginResponse	71	PTA_LoginRsp
Logout	LogoutRequest	72	PTA_LogoutReq
	LogoutResponse	73	PTA_LogoutRsp
Software Version	SCIVersionsRequest	16	PTA_VersionReq
	SCIVersionsResponse	18	PTA_SciVersionRsp
Receive File from PTA	ReceiveFileRequest	3	PTA_FileReq
	ReceiveFileResponse	4	PTA_FileRsp
	TransferProtocol	33	PTA_NextPackData
	TransferProtocol	32	PTA_NextPackReq
	TransferProtocol	34	PTA_LastPackData
	TransferProtocol	35	PTA_FileXferOverAck
Transfer File to PTA	FileRequest	3	PTA_FileReq
	FileResponse	4	PTA_FileRsp
	TransferProtocol	33	PTA_NextPackData
	TransferProtocol	32	PTA_NextPackReq
	TransferProtocol	34	PTA_LastPackData
Code Status	CodeStatusRequest	74	PTA_DMSCodeStatusReq
	CodeStatusResponse	75	PTA_DMSCodeStatusRsp

The Boot Loader executes a received command only if the PTA is logged in. Figure 3-7 shows the flow chart of the module. The PTA software should send a PTA Login command prior to send any other command. If the PTA is not logged in and sends a valid command, the Boot Loader answers with the PTA_Logout message with the status PTA not logged. Otherwise, if the command is valid, the appropriate message is sent back or the null command (if there is no answer) is sent to the PTA.

If the received packet contains error, the Packet Encoder and Decoder sends automatically the PTA_DMStoPTError response.

3.4.1.3.2 SD1.3.2 PTA_FileReq Function

The Command PTA_FileReq is used to perform a file transfer between the PTA and the Boot Loader. This command is sent by the PTA to request a memory dump or send Application code on the HW_COM port. There are only 2 types of file request recognized by the PTA Command interpreter. See Table 3-5 for the File index number and file type. The Boot Loader answer is PTA_FileRsp command.

Table 3-5: File Index Number and File Type

File Index	File type	Description	Note
0	2	Application code transferred to the Boot Loader	Reserved for communication between PTA and DMS
1	1	RAM Memory dump	Reserved for communication between PTA and DMS.

3.4.1.3.3 SD1.3.3 PTA_FileRsp Function

The command PTA_FileRsp is used to answer to the PTA_FileReq sent by the PTA.

3.4.1.3.4 SD1.3.4 PTA_VersionReq Function

The PTA_VersionReq is sent by the PTA to request the Software versions. The answer from the Boot Loader to this command is PTA_SciVersionRsp.

3.4.1.3.5 SD1.3.5 PTA_SciVersionRsp Function

The Boot Loader sends command PTA_SciVersion to answer the PTA_VersionReq sent by the PTA. If the Application is present and valid, the structure of the Application version is filled. Otherwise, the structure is left blank. See Table 3-6 for the location of SCI info to be filled into the structure.

The information regarding the version of the Application and Boot Loader is found in the Flash memory.

Table 3-6: Index of the PTA_SciVersion

Index	Name	Description
1	APP	Application
4	BOOT	Boot Loader

3.4.1.3.6 SD1.3.6 PTA_NextPackReq Function

This command is used to request a packet in the file transfer protocol. The sender can be either PTA or Boot Loader depending on the type of transferred file.

The sender is the PTA when the transferred file is the memory dump. The Boot Loader responds a PTA_NextPackData or PTA_LastPackData command with the data included in it.

The sender is the Boot Loader when the transferred file is the Application code. The Boot Loader sends this message when the previous packet was written correctly in the flash memory or at the beginning of the file transfer. The expected answer from the PTA can be PTA_NextPackData or PTA_LastPackData.

3.4.1.3.7 SD1.3.7 PTA_NextPackData Function

This command is used to send data in the file transfer process. The sender can be either PTA or Boot Loader depending on the type of transferred file.

The sender is the PTA when the transferred file is the Application code. This command is sent in answer to the Boot Loader request PTA_NextPackReq.

The sender is the Boot Loader when the transferred file is the Memory Dump file transfer. This command is sent in answer to the PTA request PTA_NextPackReq.

3.4.1.3.8 SD1.3.8 PTA_LastPackData Function

This command is used to send the last packet of data in the file transfer process. The sender can be either PTA or Boot Loader depending on the type of file transferred.

The sender is the PTA when the transferred file is the Application code. This command is sent in answer to the Boot Loader request PTA_NextPackReq when the size of the remaining data is less or equal to the maximum packet size. After recording the data included in the packet, the Boot Loader recalculates the Application code CRC.

The sender is the Boot Loader when the transferred file is the Memory Dump file transfer. This command is sent in answer to the PTA request PTA_NextPackReq when the size of the remaining data is less or equal to the maximum packet size.

3.4.1.3.9 SD1.3.9 PTA_FileXferOverAck Function

This command is used to confirm the status of the file transfer. The sender can be either PTA or Boot Loader depending on the type of file transferred.

The sender is the PTA when the transferred file is the Memory Dump file transfer. This command is sent in answer to the Boot Loader PTA_LastPackData.

The sender is the Boot Loader when the transferred file is the Application code. This command is sent in answer to the PTA PTA_LastPackData.

3.4.1.3.10 SD1.3.10 PTA_DMStoPTError Function

The Boot Loader sends this command when there is a command received from the PTA that contains errors or the command is not recognized by the Boot Loader.

3.4.1.3.11 SD1.3.11 PTA_LoginReq Function

The PTA sends this command to log into the Boot Loader. The PTA should send this command if a delay of more than 60 sec elapsed since the last valid command received by the Boot Loader. The Boot Loader sends the PTA_LoginRsp answer. The login string to specific project is defined in the specific project's *SACS* document.

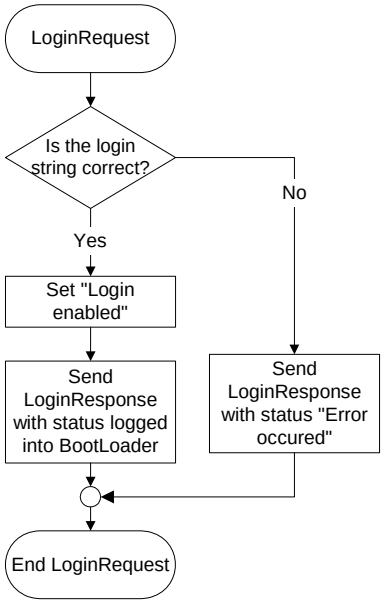

Figure 3-8: PTA Login Request

3.4.1.3.12 SD1.3.12 PTA_LoginRsp Function

The PTA_LoginRsp is sent by the Boot Loader to confirm the success of the login request. The code for login is displayed in Table 3-7.

Table 3-7: Codes of the Login and Logout

Codes	Description
0	Not logged in
1	Login error
2	Boot Loader Login/Logout
3	Application Login/Logout
4	Boot Loader Login/Logout Without Application code present

3.4.1.3.13 SD1.3.13 PTA_LogoutReq Function

The PTA sends this command to logout from the Boot Loader. The Boot Loader sends the PTA_Logout answer.

3.4.1.3.14 SD1.3.14 PTA_LogoutRsp Function

The PTA_LogoutRsp is sent by the Boot Loader to confirm the success of the logout request. The code for logout is displayed in Table 3-7.
The Boot Loader will send this command with the status code 0 if the PTA was not logged in before or is not logged anymore because of the 60 sec timeout elapsed.

3.4.1.3.15 SD1.3.15 PTA_DMSCodeStatusReq Function

The PTA sends the PTA_CodeStatusReq to the Boot Loader.

Table 3-8: Index of the CodeStatusRequest

Index	Name	Description
1	APP	Application
4	BOOT	Boot Loader

3.4.1.3.16 SD1.3.16 PTA_DMSCodeStatusRsp Function

The Boot Loader sends this command PTA_DMSCodeStatusRsp to the PTA in answer to the PTA_DMSCodeStatusReq.

If the Application is present and valid, the structure of the Application code status is filled. Otherwise, the structure is left blank.

The information regarding the code status of the Application and Boot Loader is found in the Non Volatile memory.

3.4.1.3.17 SD1.3.17 PTA_ComTestReq Function

The PTA_ComTestReq is sent by the PTA to keep communication active (avoid timeout).

3.4.1.3.18 SD1.3.18 PTA_ComTestRsp Function

The Boot Loader responds back to the request PTA_ComTestReq with PTA_ComTestRsp.

3.4.1.4 SD1.4 Watchdog Module

The Watchdog Task is used to send a still alive signal to the watchdog circuit. A change in the state of the output pin is performed each time the module is executed. The period of signals sent to the watchdog should be less than 0.1 second. This module is called from the Boot Task and it also toggles the Warning LED.

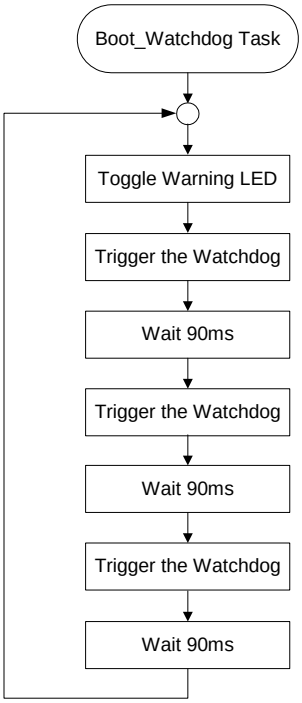

Figure 3-9: WDT_Refresh flow chart

3.4.1.5 SD1.5 LED Utility Module

The LED Utility Module is responsible for displaying the fault status with the fault LED. The module receives a command corresponding to the fault code and performs the associated action. See Table 3-9 for a list of command codes received by the module. The module is used to turn ON, turn OFF and perform a blink pattern specific to the command code. In blink mode, the LED is kept ON 250 ms and a inter blink delay of 500 ms. After the last blink of the pattern, a delay of 1 second is inserted while the LED is maintained OFF. The pattern is then re-started. See Appendix A for specific fault code flashing pattern.

Table 3-9: Command codes sent to Fault LED Utilities

Status Code (n)	Description
0	Turn OFF Fault LED
1	Turn ON Fault LED
2	Blink once the LED and repeat after one second.
3	Blink twice the LED and repeat after one second.
4	Blink three times the LED and repeat after one second.
5	Blink four the LED and repeat after one second.
6	Blink five times the LED and repeat after one second.
7	Blink six times the LED and repeat after one second.
8 and More	Blink n - 1 times the LED and repeat after one second.

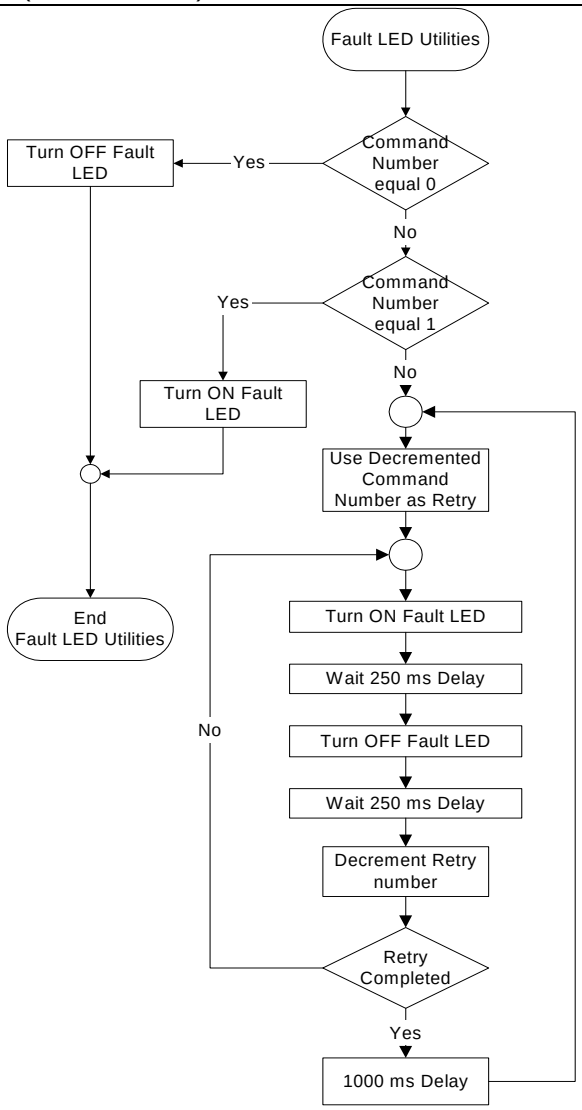

Figure 3-10: FaultLEDutility Flow Chart

3.4.1.6 SD1.6 Memory Integrity Utility Module

3.4.1.6.1 SD1.6.1 Internal RAM Integrity Check

The purpose of the Internal RAM integrity check module is to verify sequentially every cell of the internal memory in the CPU and verifies that each address line of the internal bus is responding. The Internal RAM integrity test is performed after power up before services, peripherals or interruptions can be allowed in the CPU. If successful the Boot Loader allows the CPU to use the entire Internal RAM, including the area of the CPU stack.

The internal RAM is divided into two different sections, XRAM and IRAM. Figure 3-11 shows the flow chart of the internal RAM integrity check.

The first step is to verify each address lines separately to confirm that the internal address and data bus is working properly.

This task is achieved by writing in each address line a sequential number. The data is then read back and is verified that the data is still valid. If the comparison fails, the module exits and the fault LED blinks once each second. See Appendix A for description of faults. This test ensures that every address line is responding. If the test is successful, the system starts the pattern test that verifies the memory.

The second test consists of a pattern test. This test writes the data 0xAA55 and then reads it back. If there is no error in the same memory cell, the data 0x55AA written and is then read back. The entire XRAM is then tested using the same pattern. In case of an error, the module exits and the fault LED blinks once each second. See Appendix A for description of faults. If the test on the XRAM is successful, the system starts to verify the IRAM section beginning with the address/data lines test followed by the pattern test.

The memory address for IRAM and XRAM is defined at the specific project's *SACS* document.

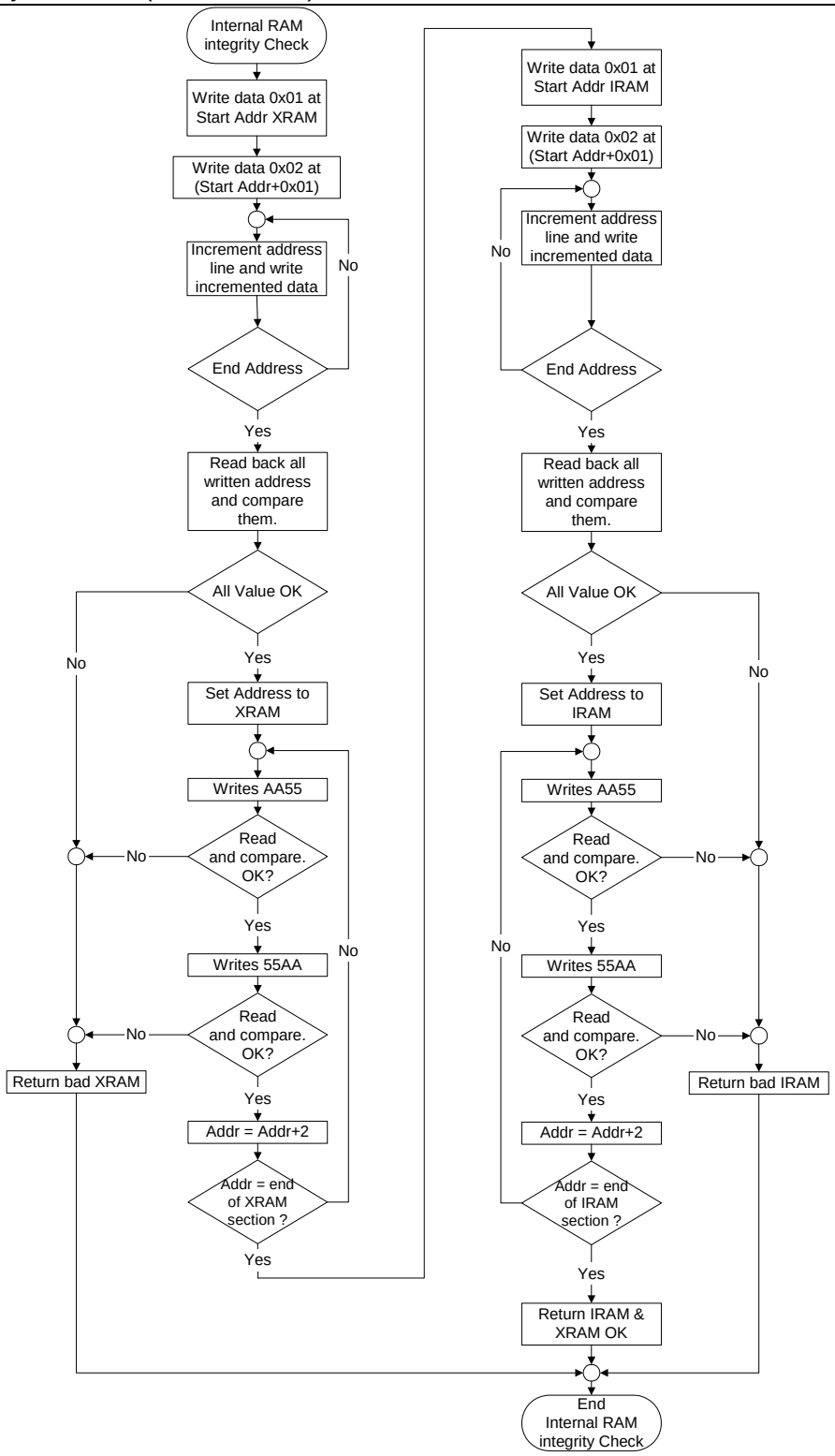

Figure 3-11: IRAM InternalRAMCheck flow chart

3.4.1.6.2 SD1.6.2 Volatile and Non-Volatile Memory Integrity Check

The RAM Integrity Check Module needs to preserve the integrity of the memory under test and has the capability of data recovery. The RAM Integrity Check Module keeps a backup of data that have to be tested to maintain the integrity of the RAM. The Volatile memory contains temporary data that is reset upon each reboot. The Non-Volatile contains data valuable to the Application and Boot Loader. The complete RAM array is seen as a continuous RAM for Boot Loader.

Verification is performed at startup of the module to verify if a previous session was in progress. It returns the data to their respective address if another session was in progress before the module starts the tests. The second step is to verify the integrity of the memory location where data is backed-up; pattern data 0xAA55 is written and then compared and pattern data 0x55AA is written and compared. At this point, if an error occurs, the module exits and call LED_FaultLed Module with the fail code.

3.4.1.6.2.1 Address / Data lines test

The first step is to verify that each address/data lines are working properly. The test writes a sequential number at each address line. The sequential number is then read back and compared to sequential values. The second test is to verify the memory.

Both memory devices (Volatile and Non-Volatile memories) need to be tested independently; all address lines are tested. This test ensures that every address/data lines are responding. If the test is successful, the pattern test is started.

Before the module starts to write sequential number, a backup of written address data is performed to ensure there is no loss of data in case of power failure. At the end of the test, data backed up is restored. If power failure occurs before the end of the test, the module restores the data at next start-up to preserve the integrity of the Non-Volatile memory.

Table 3-10: Memory Test Structure

Address	Data
Start Address (SA)	0x01
SA + 2^2	0x02
SA + 2^3	0x03
SA + 2^4	0x04
SA + 2^5	0x05
…….	…….
SA + $2^{(n-1)}$	n-1
SA + 2^n	n

Table 3-1 shows the structure recorded in memory when address lines verification test is started. The first step of the module is to read the data in memory and create a copy of the data (ucData) along with the number of the data (ucNbData). Only after all the data have been copied, the command 0x01 (ucCommand) and one's complement (ucComplCommand) of the commands are written. Then the CRC is calculated on the first two bytes (command and complement) and is saved (uiCrc). To indicate that the test has been in progress, complements of the command and the CRC are written. This security is used in case of power failure.

The sequential numbers are written in memory as per data shown in Table 3-10. The sequential numbers are then read back and compared to the sequence. In case of error during the pattern, the module exits and call LED_FaultLed Module with the fault code.

After the test is completed and successful, the data is then restored to all address location and the command is erased. The value of the command is returned to value 0x00.

3.4.1.6.2.2 Pattern test

The purpose of the pattern test is to test the memory. Table 3-1 shows the structure recorded in memory to preserve the integrity of the memory when pattern test is running. The test consists in writing two different patterns in every cell.

The first step of the module is to read the data of the memory cell under test and create a copy of the data to memory backup are (as seen in the Table 3-1). The data read is an integer type and written into variable uiData. Only after all the data have been copied, the command 0x02 (ucCommand) is written and the complement's one of the commands at ucComplCommand. A CRC is calculated on the first two bytes (command and complement) and written into uiCrc. To indicate that the test has been in progress, complements of the command and the CRC are written. This security is used in case of power failure.

After the memory cell under test is completed and successful, the data is then restored to the address location and then the command is erased. The ucCommand is returned to value 0x00.

The pattern test writes data 0xAA55 in the memory cell under test and then to read it back to compare the results. In the same memory cell, data 0x55AA is written and then read back for comparison. In case of error during the pattern test, the module exits and calls LED_FaultLed module with the fault status.

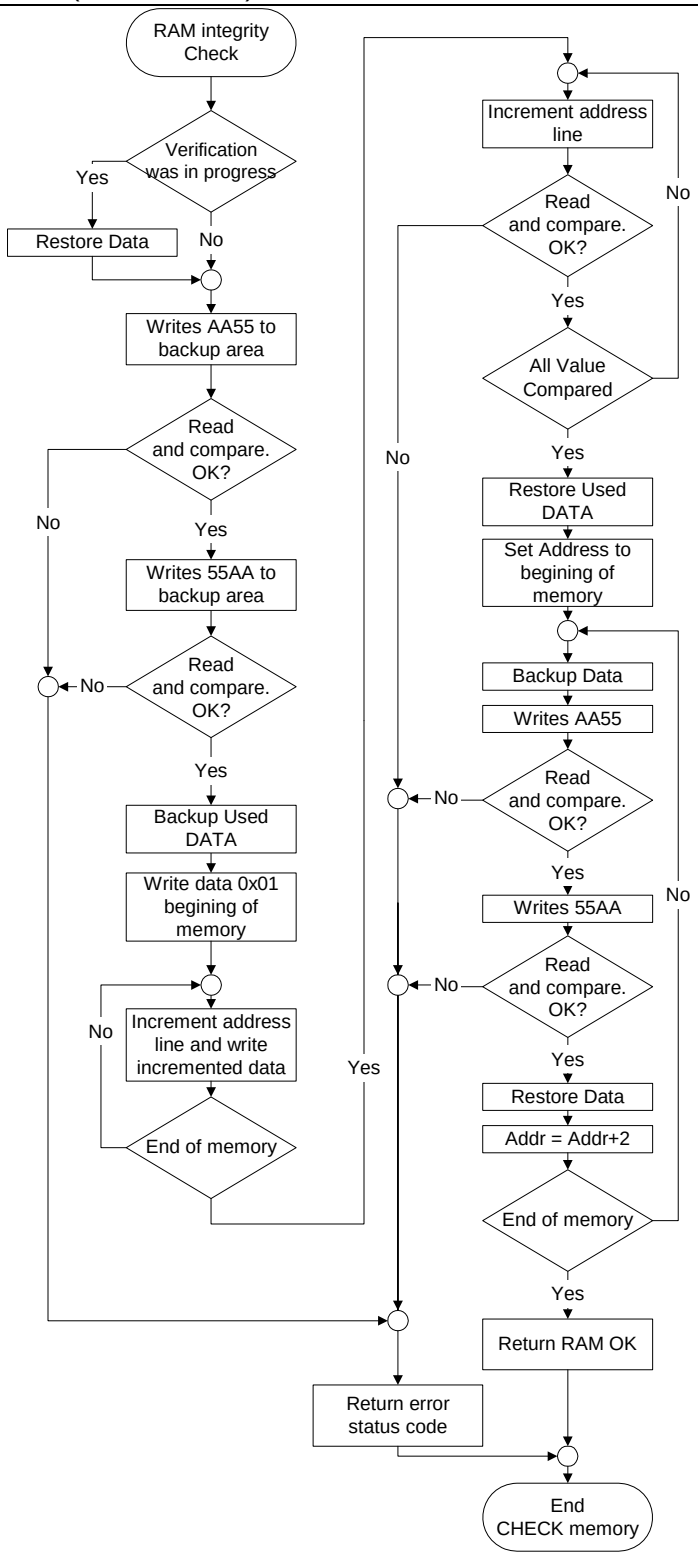

Figure 3-12: RAM_RamCheck flow chart

3.4.1.7 SD1.7 CRC Utility Module

The CRC Validation Module verifies the integrity of the Boot Loader and Application code by calculating the CRC on the code stored in Flash memory. The CRC, code length, and revision number are included in the Boot Loader code and the Application code at a specific offset/location. The CRC Validation Module expects to find the CRC, code size, Major revision number, Minor revision number, and revision date at these specific address in the Flash Memory.

The CRC is calculated according to the CRC-CCITT Standard. The initial value used in any cases to calculate the CRC is zero. The Table 3-11 and Table 3-12 describe the CRC computation algorithm and look up table.

Table 3-11: Function to calculate the CRC16

```
/*****************************************************************************/
//  PURPOSE         : This function calculates the CRC16 of the DATA received.
//  INPUT(S)        : ucNewData = Is the new data to add on the old crc. The initial
//                                value to calculate crc is 0.
//                    uiOldCrc  = Is the old crc value to add.
//  RETURN          : The new crc calculated with the new data received.
/*****************************************************************************/
unsigned int CRC_UpdtCrc (unsigned char ucNewData, unsigned int uiOldCrc)
{
    return (crctab[(uiOldCrc >> 8)^ucNewData] ^(uiOldCrc << 8));
}
```

Table 3-12: CRC Look up Table for CRC16 calculation

```
const unsigned int near crctab[] = {
     0x0000,  0x1021,  0x2042,  0x3063,  0x4084,  0x50A5,  0x60C6,  0x70E7,
     0x8108,  0x9129,  0xA14A,  0xB16B,  0xC18C,  0xD1AD,  0xE1CE,  0xF1EF,
     0x1231,  0x0210,  0x3273,  0x2252,  0x52B5,  0x4294,  0x72F7,  0x62D6,
     0x9339,  0x8318,  0xB37B,  0xA35A,  0xD3BD,  0xC39C,  0xF3FF,  0xE3DE,
     0x2462,  0x3443,  0x0420,  0x1401,  0x64E6,  0x74C7,  0x44A4,  0x5485,
     0xA56A,  0xB54B,  0x8528,  0x9509,  0xE5EE,  0xF5CF,  0xC5AC,  0xD58D,
     0x3653,  0x2672,  0x1611,  0x0630,  0x76D7,  0x66F6,  0x5695,  0x46B4,
     0xB75B,  0xA77A,  0x9719,  0x8738,  0xF7DF,  0xE7FE,  0xD79D,  0xC7BC,
     0x48C4,  0x58E5,  0x6886,  0x78A7,  0x0840,  0x1861,  0x2802,  0x3823,
     0xC9CC,  0xD9ED,  0xE98E,  0xF9AF,  0x8948,  0x9969,  0xA90A,  0xB92B,
     0x5AF5,  0x4AD4,  0x7AB7,  0x6A96,  0x1A71,  0x0A50,  0x3A33,  0x2A12,
     0xDBFD,  0xCBDC,  0xFBBF,  0xEB9E,  0x9B79,  0x8B58,  0xBB3B,  0xAB1A,
     0x6CA6,  0x7C87,  0x4CE4,  0x5CC5,  0x2C22,  0x3C03,  0x0C60,  0x1C41,
     0xEDAE,  0xFD8F,  0xCDEC,  0xDDCD,  0xAD2A,  0xBD0B,  0x8D68,  0x9D49,
     0x7E97,  0x6EB6,  0x5ED5,  0x4EF4,  0x3E13,  0x2E32,  0x1E51,  0x0E70,
     0xFF9F,  0xEFBE,  0xDFDD,  0xCFFC,  0xBF1B,  0xAF3A,  0x9F59,  0x8F78,
     0x9188,  0x81A9,  0xB1CA,  0xA1EB,  0xD10C,  0xC12D,  0xF14E,  0xE16F,
     0x1080,  0x00A1,  0x30C2,  0x20E3,  0x5004,  0x4025,  0x7046,  0x6067,
     0x83B9,  0x9398,  0xA3FB,  0xB3DA,  0xC33D,  0xD31C,  0xE37F,  0xF35E,
     0x02B1,  0x1290,  0x22F3,  0x32D2,  0x4235,  0x5214,  0x6277,  0x7256,
     0xB5EA,  0xA5CB,  0x95A8,  0x8589,  0xF56E,  0xE54F,  0xD52C,  0xC50D,
     0x34E2,  0x24C3,  0x14A0,  0x0481,  0x7466,  0x6447,  0x5424,  0x4405,
     0xA7DB,  0xB7FA,  0x8799,  0x97B8,  0xE75F,  0xF77E,  0xC71D,  0xD73C,
     0x26D3,  0x36F2,  0x0691,  0x16B0,  0x6657,  0x7676,  0x4615,  0x5634,
     0xD94C,  0xC96D,  0xF90E,  0xE92F,  0x99C8,  0x89E9,  0xB98A,  0xA9AB,
     0x5844,  0x4865,  0x7806,  0x6827,  0x18C0,  0x08E1,  0x3882,  0x28A3,
     0xCB7D,  0xDB5C,  0xEB3F,  0xFB1E,  0x8BF9,  0x9BD8,  0xABBB,  0xBB9A,
     0x4A75,  0x5A54,  0x6A37,  0x7A16,  0x0AF1,  0x1AD0,  0x2AB3,  0x3A92,
     0xFD2E,  0xED0F,  0xDD6C,  0xCD4D,  0xBDAA,  0xAD8B,  0x9DE8,  0x8DC9,
     0x7C26,  0x6C07,  0x5C64,  0x4C45,  0x3CA2,  0x2C83,  0x1CE0,  0x0CC1,
     0xEF1F,  0xFF3E,  0xCF5D,  0xDF7C,  0xAF9B,  0xBFBA,  0x8FD9,  0x9FF8,
     0x6E17,  0x7E36,  0x4E55,  0x5E74,  0x2E93,  0x3EB2,  0x0ED1,  0x1EF0
};
```

Figure 3-13 shows the flow chart of the module that verifies the CRC of code.

At first, the modules verifies if the code is present by reading the length at address shown in Table 3-2, and verifies if the value is greater than zero. When the value read is zero or 0xFFFFFF for the code size, the execution of the module is stopped and a status code is returned to inform that the code under test is not present in Flash Memory.

If the length is greater than 0, the CRC is calculated for the length of the code except for the location where the CRC itself is stored. The CRC calculation does not include the CRC itself. When the CRC calculation is finished, the calculated CRC is then compared to the value stored in Flash memory and the module exits with the status code. See Table 3-13 for status code description.

Table 3-13: Status Codes returned for CRC Validation Module

Status Code	Description
0	No error, CRC match.
1	Code not present in memory.
2	Error, CRC doesn't match.
3	Other error.

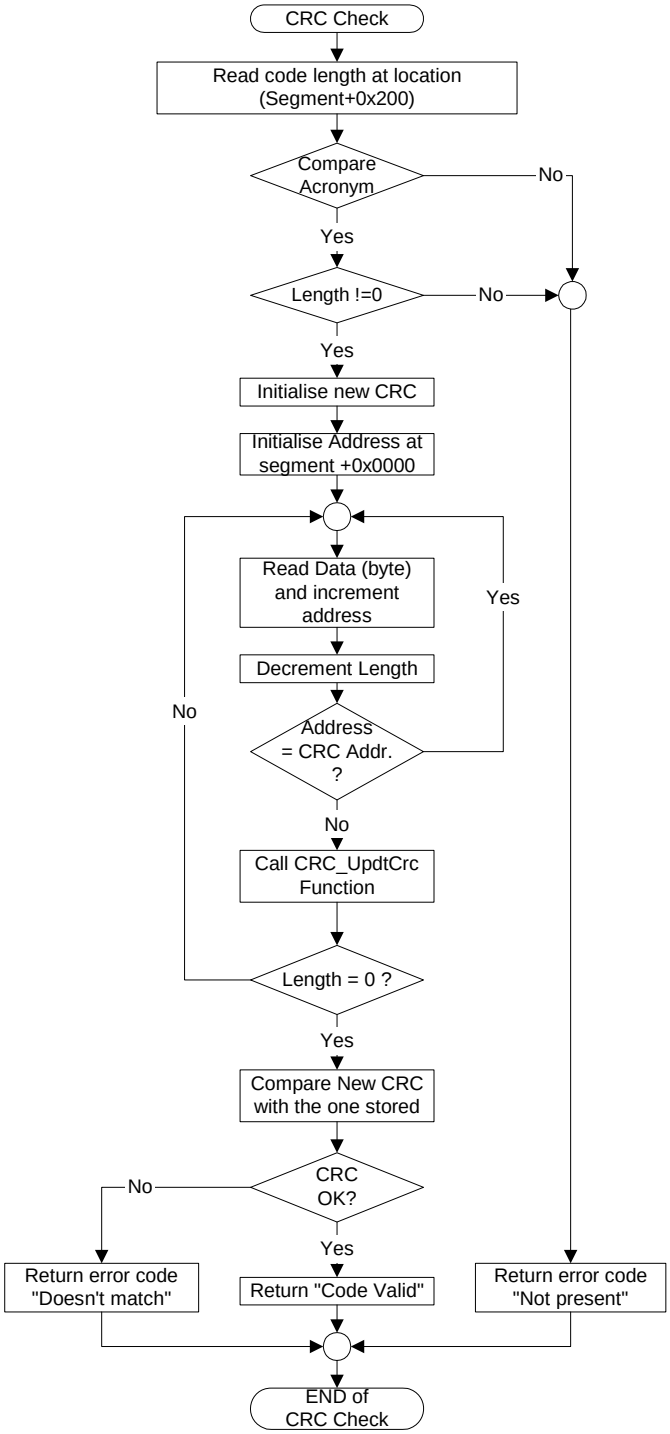

Figure 3-13: CRC_CodeIntegrity flow chart

3.4.1.8 SD1.8 Flash Utility Module

The Flash Memory Utilities contains two functions: FLASH_EraseAppCode and FLASH_WritePacket. The FLASH_EraseAppCode erases sector 2 to 7 of the Flash memory. Those sectors contain the Application code. The FLASH_WritePacket writes packet of data received from the PTA. It is used to upload a new Application.

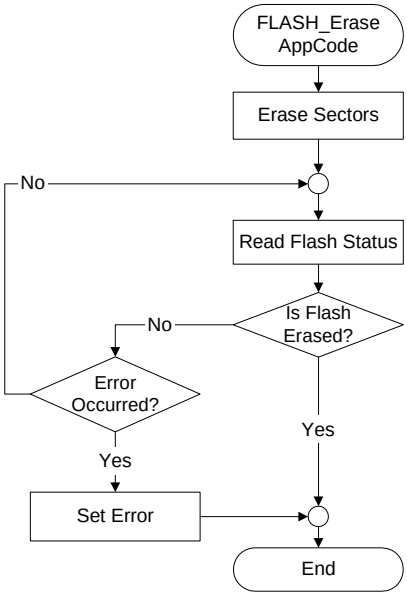

Figure 3-14: FLASH_EraseAppCode Flow Chart

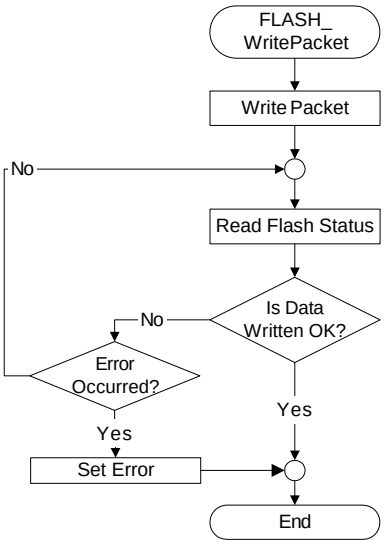

Figure 3-15: FLASH_WritePacket Flow Chart

3.4.2 Data Entities Detailed Design

See section 3.4.1 for data description.

3.5 Operating System Services

The used Operating System is a Real-Time Operating System intended for embedded applications.
The task scheduling is configured to ensure that the CPU usage never goes over 50%.

The Operating System services used by the Boot Loader are:

- Task Manager
- Semaphore
- Message Queue

See specific project's *SACS* document for the OS information.

3.6 Memory Layout

The memory layout limits the usage of each type of device (code flash memory, volatile data memory, non-volatile data memory) to 70%. See specific project's *SACS* document for the memory layout.

APPENDIX A: FAULT LED BLINKING PATTERNS

This Appendix A lists the faults that can occur in the Boot Loader and displayed with the Fault LED.

Fault Code	Fault Name	Fault Description	Fault LED Flash pattern
0	No Error	No error on the system.	The Fault LED is off.
1	Application CRC32 error	The CRC32 of the Application is not valid	The Fault LED is on.
2	IRAM error	Memory location or address line in the IRAM is defective	One blink of 250 ms of the Fault LED, repeated after 1000 ms.
3	Boot Loader CRC32 error	The CRC32 of the Boot Loader is not valid	Two Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
4	Internal Volatile Memory Address error	Address line on SRAM is defective	Three Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
5	Internal Volatile Memory Pattern error	Memory location on SRAM is defective	Four Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
6	Non-Volatile Address error	Address line on external NVRAM is defective	Five Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
7	Non-Volatile Pattern error	Memory location on NVRAM is defective	Six Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
8	External Volatile Address error	Address line on External SRAM is defective	Seven Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.
9	External Volatile Pattern error	Memory location on External SRAM is defective	Eight Blinks of 250 ms of the Fault LED and an inter-blink delay of 500 ms. A delay of 1000 ms before re-cycles.

APPENDIX B: SPECIFIC PROJECT DESIGN ELEMENT

The following items are defined in the specific project's *Software Applications Configuration Specifications (SACS).*

B.1: PTA Login String

There are two login strings: the login request string from the PTA and the login response string from the Boot Loader.
Both strings are defined as following in the *SACS* document*:*

- The PTA login request string is "[PROJECT_NAME], PTA Door Module System, VSRS"
- The PTA login response string is "[PROJECT_NAME], Door Module System, VSRS"

B.2: Operating System Information

The OS information is defined in the *SACS* document as following:

- OS Company Name (ex: Keil Elektronik)
- OS Name (ex: Keil ARTX-166 Real-Time OS)
- OS Version Number (ex: Version 2.02a)

B.3: Memory Address

The memory address used for volatile and non-volatile is defined in the *SACS* document as following:

- Type of the memory (ex: IRAM)
- The address of the memory (ex: IRAM is assigned to 0xF600 ~ 0xFDFF)
- The size of the memory (ex: IRAM has 2Kbyes)

B.4: Code Information Address

The addresses of code information for the Boot Loader and the Application are defined in the *SACS* document:

- The address of the Boot Loader code information (ex: 0x0200h)
- The address of the Application code information (ex: 0x020200h)

B.5: Memory Layout

The memory layout is defined in the *SACS* document as a following table format:

Memory Mapping on the Door Module System Unit

Start Address	End Address	Size	Description
[Address in hex]	[Address in hex]	[size in KB]	[Description]
…	…	…	…
Ex: 0x000000	Ex: 0x007FFF	Ex: 32 KB	Ex: Boot Loader Flash Code

Static RAM Memory Mapping

Start Address	End Address	Class	Size	Description
[Address in hex]	[Address in hex]	[Class Name]	[size in KB]	[Description]
…	…	…	…	…
Ex: 0x104000	Ex: 0x107FFF	Ex: HOSSTK	Ex: 16.00 KB	Ex: OS Stack